

ALGORITHMS 1 – LAB

The Hash Table ADT

MARCO GUAZZONE

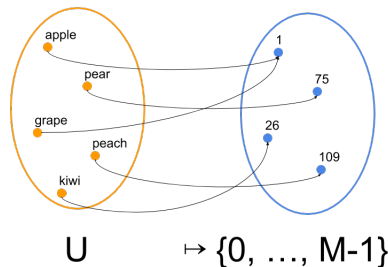
DiSIT, University of Piemonte Orientale
`marco.guazzone@uniupo.it`

The Hash Table ADT

In the search problem, is it possible to get $O(1)$ complexity, on average?

- If we had integer keys, we could use an array to store values and use keys as indices
- Idea: why don't we transform keys into integer numbers?

Hash Table is a data structure used to map keys to values that is based on this idea



The Hash Table ADT

Practical applications:

- Searching problems
- Symbol tables for compilers
- Cache systems
- Peer-to-Peer computing (see Distributed Hash Tables)
- Implementation of different ADTs
 - Sets
 - Dictionaries
- ...

Complexity Summary for Dictionary Implementations

ADT	Worst-case Time		Average-case Time		Space	Sorted Keys?
	search	insert	search	insert		
BST	$O(N)$	$O(N)$	$O(\log N)$	$O(\log N)$	$O(N)$	Yes
Hash Table	$O(\log N)$	$O(\log N)$	$O(1)$	$O(1)$	$O(N + M)$	No

(where N is the number of stored keys and M is the capacity of the Hash Table)

Hash Tables are bad when we are interested in getting keys in order (e.g., to find min or max key, or to retrieve the keys inside a given range).

NOTE: The space complexity for Hash Table is $\Theta(M)$ if open addressing is used.

Definitions

- **Hashing** is the process of mapping a key space U to integer values $\{0, \dots, M - 1\}$ representing positions in a table (i.e., the **hash table**)
- **Hash function**: a function $h(\cdot)$ that transforms a key into an integer value

$$h : U \rightarrow \{0, \dots, M - 1\}$$

We also say that $h(k)$ is the **hash value** of key k

The Hash Table ADT

Definitions (cont'd)

A **Hash Table** (HT) is a key-indexed array of M cells (called **slots** or **buckets**) where indices are computed by the hash function $h(\cdot)$ and values (as well as keys) are stored in the cells of the array

Contact (Key)	Phone Number (Value)
Luke Skywalker	555 – 1234
Darth Vader	555 – 8976
Han Solo	555 – 5030

$$h(\text{"Luke Skywalker"}) = 2$$

...

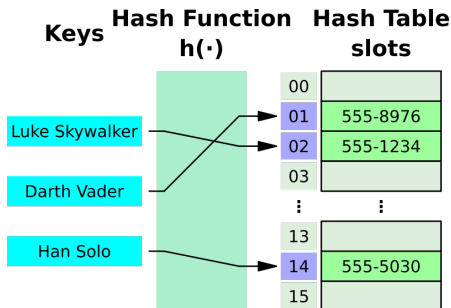


Image source: Inspired by "Hash table 3 1 1 0 1 0 0 SP" by Jorge Stolfi – Own work. Licensed under CC BY-SA 3.0 via

Wikimedia Commons – https://commons.wikimedia.org/wiki/File:Hash_table_3_1_1_0_1_0_0_SP.svg

Definitions (cont'd)

- **load factor**: the number of stored elements divided by the number of slots

$$\alpha = \frac{N}{M}$$

where:

- N : number of stored elements in the HT
- M : number of slots of the HT
- The larger α , the slower the operations on the HT $\rightarrow M$ underestimated
- The smaller α , the greater the waste of memory $\rightarrow M$ overestimated

Fundamental Operations

- `put( $t, k, v$ )`: inserts/replaces the value v identified by k in t and returns the previous value (if any)
- `insert( $t, k, v$ )`: inserts the value v identified by k in t only if it isn't a duplicate
- `get( $t, k$ )`: returns the value v identified by k in t
- `contains( $t, k$ )`: tells if a value identified by k exists in t
- `delete( $t, k$ )`: removes the key-value pair identified by k from t
- `clear( $t$ )`: removes all the key-value pairs from t
- `is_empty( $t$ )`: tells whether t is an empty hash table or not
- `size( $t$ )`: returns the number of stored keys
- `capacity( $t$ )`: returns the number of slots
- `load_factor( $t$ )`: returns the load factor of t

Challenges

- ❶ How to implement a “good” hash function?
- ❷ How to cope with the situation where different keys hash to the same array index?

Challenge #1: Implement a Good Hash Function

Requirements in implementing a good hash function:

- *Consistency*: equal keys must produce the same hash value
- *Efficiency*: the hash value should be efficient to compute
- *Uniformity*: the hash function should uniformly distribute the keys

Satisfying these requirements simultaneously is a job for experts!

This topic is out of the scope of this course!

Challenge #1: Implement a Good Hash Function (cont'd)

Example of hash functions for integers:

- **Division method** (aka **modular hashing**):

$$h(k) = k \bmod M$$

- **Multiplication method:**

$$h(k) = \lfloor M \cdot (a \cdot k \bmod 1) \rfloor$$

where:

- $a \in (0, 1)$ (Knuth suggested: $a = (\sqrt{5} - 1)/2$)
- $(x \bmod 1)$ means the fractional part of x (e.g., $11.4 \bmod 1 = 0.4$)
- $\lfloor x \rfloor$ means the floor of x (i.e., the greatest integer $\leq x$)

Challenge #1: Implement a Good Hash Function (cont'd)

Example of hash functions for strings:

- Given a string $k = (k_0, \dots, k_{\ell-1})$ represented as an array of ℓ characters

$$h(k) = g(k, \ell - 1),$$

$$g(k, i) = \begin{cases} (h_0 \cdot a + \text{INT}(k_0)) \bmod M, & i = 0, \\ (g(k, i-1) \cdot a + \text{INT}(k_i)) \bmod M, & 0 < i < \ell. \end{cases}$$

where:

- $\text{INT}(c)$ is the integer representation of character c
- h_0 is an initial value
- a is a multiplicative constant

Challenge #1: Implement a Good Hash Function (cont'd)

EXAMPLE: Hashing the String "car"

Let $k = \text{"car"}$ and $\text{INT}(c) = \langle \text{the ASCII code of character } c \rangle$, then

$$\begin{aligned} h(k) &= (((((h_0 + \text{INT}('c')) \bmod M) \cdot a + \text{INT}('a')) \bmod M) \cdot a + \text{INT}('r')) \bmod M, \\ &= (((((h_0 + 99) \bmod M) \cdot a + 97) \bmod M) \cdot a + 114) \bmod M. \end{aligned}$$

If $h_0 = 0$, $a = 31$, and $M = 1$, then:

$$\begin{aligned} h(k) &= (99 \cdot 31 + 97) \cdot 31 + 114, \\ &= 98260. \end{aligned}$$

This is the method used by the Java programming language for hashing strings.

Challenge #1: Implement a Good Hash Function (cont'd)

Examples of string hashing implementation:

Implementation	h_0	a
D.J. Bernstein's djb2	5381	33
Kernighan & Ritchie's C book (2 nd Ed.)	0	31
<code>java.lang.String.hashCode()</code>	0	31
SGI STL's <code>hash_fun</code>	0	5
STLport's <code>hash_fun</code>	0	33

Challenge #2: Collision Resolution

- In a perfect world, hash functions generate a different index for every key
- However, in practice $|U| \gg M$ and hence **collisions** are inevitable

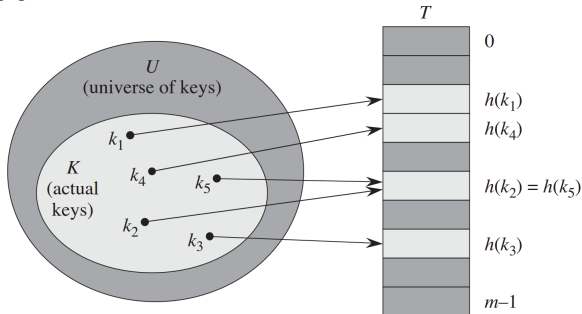


Image source: T.H. Cormen et al. "Introduction to Algorithms, 3/E." The MIT Press, 2009.

Challenge #2: Collision Resolution (cont'd)

Two general approaches:

- **Separate chaining**
 - Also known as *closed addressing* or *open hashing*
- **Open addressing**
 - Also known as *closed hashing*

Other **hybrid approaches** (that combine open and closed addressing) have been proposed

Separate Chaining

- Collisions are stored in a linked list associated with each hashing value (i.e., slot index)
- Each slot of the HT stores a pointer to the head of the associated list of collisions, where each node stores a colliding key and its associated value
- If the HT has capacity M and stores N keys, the average length of the list is N/M (i.e., the load factor α)
→ Number of compares for search miss and insert is $\sim N/M$

Separate Chaining (cont'd)

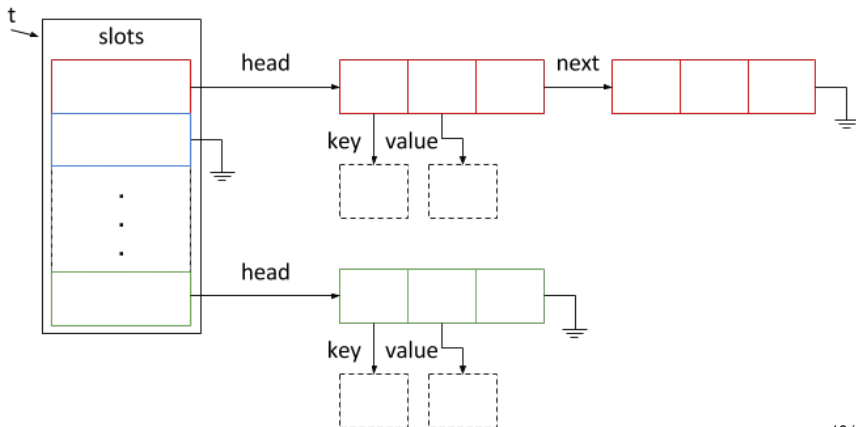
EXAMPLE: Search for key k

- 1 Compute $h(k)$
- 2 Scan the list at index $h(k)$ of the HT until either key k is found or the end of list is reached

Separate Chaining (cont'd)

Representation of an HT with Separate Chaining

- slots is an array of entries, each of which points to a list of colliding keys (head)
- Every list node contains a pointer to the **key**, to the **value** and to the **next** list node



Separate Chaining (cont'd)

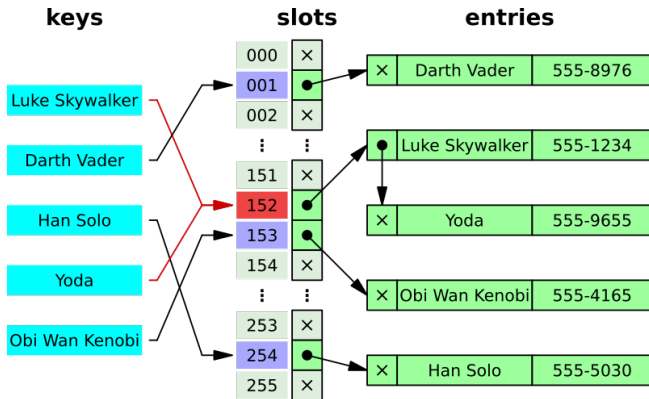


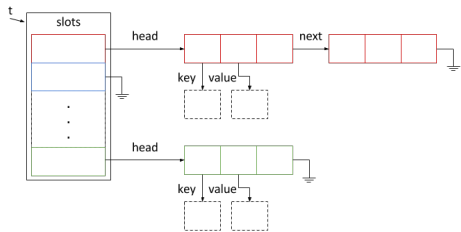
Image source: Inspired by "Hash table 5 0 1 1 1 1 1 LL" by Jorge Stolfi – Own work. Licensed under CC BY-SA 3.0 via

Wikimedia Commons – https://commons.wikimedia.org/wiki/File:Hash_table_5_0_1_1_1_1_1_LL.svg

Separate Chaining (cont'd)

Let t be an HT,

- $t[j]$: the slot of t at index j
- $\text{CAPACITY}(t)$: the total number of slots in t
- $\text{LOADFACTOR}(t)$: the load factor of t
- $\text{SIZE}(t)$: the number of stored keys in t
- $\text{HEAD}(t[j])$: the head of the list of colliding keys at slot j
- $\text{NEXT}(n)$: the node in the list that follows node n
- $\text{KEY}(n)$: the key stored in node n
- $\text{VALUE}(n)$: the value stored in node n
- $\text{HASH}(k)$: the hash value of k



Separate Chaining (cont'd)

The $\text{get}(t, k)$ operation: searches a key and returns the associated value

```
1: function GET( $t, k$ )
2:    $h \leftarrow \text{HASH}(k)$ 
3:    $n \leftarrow \text{HEAD}(t[h])$ 
4:   while  $n \neq \text{null}$  and  $k \neq \text{KEY}(n)$  do
5:      $n \leftarrow \text{NEXT}(n)$ 
6:   end while
7:   if  $n \neq \text{null}$  then
8:     return  $\text{VALUE}(n)$ 
9:   else
10:    return null
11:  end if
12: end function
```

Separate Chaining (cont'd)

The `contains(t,k)` operation: like the `get(t,k)` operation, but returns true or false

```
1: function CONTAINS( $t, k$ )
2:    $h \leftarrow \text{HASH}(k)$ 
3:    $n \leftarrow \text{HEAD}(t[h])$ 
4:   while  $n \neq \text{null}$  and  $k \neq \text{KEY}(n)$  do
5:      $n \leftarrow \text{NEXT}(n)$ 
6:   end while
7:   if  $n \neq \text{null}$  then
8:     return true
9:   else
10:    return false
11:  end if
12: end function
```

Separate Chaining (cont'd)

The $\text{put}(t, k, v)$ operation: inserts a new key-value pair or updates the value of an existing key, and returns the previously stored value (in case of an update) or `null` (in case of an insertion)

```
1: function PUT( $t, k, v$ )
2:    $V_{old} \leftarrow \text{null}$ 
3:    $h \leftarrow \text{HASH}(k)$ 
4:    $n \leftarrow \text{HEAD}(t[h])$ 
5:   while  $n \neq \text{null}$  and  $k \neq \text{KEY}(n)$  do
6:      $n \leftarrow \text{NEXT}(n)$ 
7:   end while
8:   if  $n = \text{null}$  then
9:      $n \leftarrow \text{CREATE\_NODE}()$ 
10:     $\text{KEY}(n) \leftarrow k$ 
11:     $\text{VALUE}(n) \leftarrow v$ 
12:     $\text{NEXT}(n) \leftarrow \text{HEAD}(t[h])$ 
13:     $\text{HEAD}(t[h]) \leftarrow n$ 
14:   else
15:      $V_{old} \leftarrow \text{VALUE}(n)$ 
16:      $\text{VALUE}(n) \leftarrow v$ 
17:   end if
18:   return  $V_{old}$ 
19: end function
```

Separate Chaining (cont'd)

The $\text{insert}(t, k, v)$ operation: like the $\text{put}(t, k, v)$ operation, but ignores duplicates

```
1: procedure INSERT( $t, k, v$ )
2:    $h \leftarrow \text{HASH}(k)$ 
3:    $n \leftarrow \text{HEAD}(t[h])$ 
4:   while  $n \neq \text{null}$  and  $k \neq \text{KEY}(n)$  do
5:      $n \leftarrow \text{NEXT}(n)$ 
6:   end while
7:   if  $n = \text{null}$  then
8:      $n \leftarrow \text{CREATE\_NODE}( )$ 
9:      $\text{KEY}(n) \leftarrow k$ 
10:     $\text{VALUE}(n) \leftarrow v$ 
11:     $\text{NEXT}(n) \leftarrow \text{HEAD}(t[h])$ 
12:     $\text{HEAD}(t[h]) \leftarrow n$ 
13:   end if
14: end procedure
```

Separate Chaining (cont'd)

The `delete(t,k)` operation: removes a key and the associated value

```
1: procedure DELETE( $t, k$ )
2:    $h \leftarrow \text{HASH}(k)$ 
3:    $n \leftarrow \text{HEAD}(t[h])$ 
4:    $p \leftarrow \text{null}$ 
5:   while  $n \neq \text{null}$  and  $k \neq \text{KEY}(n)$  do
6:      $p \leftarrow n$ 
7:      $n \leftarrow \text{NEXT}(n)$ 
8:   end while
9:   if  $n \neq \text{null}$  then
10:    if  $p = \text{null}$  then
11:       $\text{HEAD}(t[h]) \leftarrow \text{NEXT}(n)$ 
12:    else
13:       $\text{NEXT}(p) \leftarrow \text{NEXT}(n)$ 
14:    end if
15:     $\text{DESTROYNODE}(n)$ 
16:  end if
17: end procedure
```

Open Addressing

- **Open Addressing**: collisions are stored directly in the HT (in its empty slots)
 - Probes slots by means of a new hash function $c(k, i)$, for $i = 0, 1, \dots$
 - $c(k, i)$ internally uses the hash function $h(k)$ (see next slides)
 - Stores a colliding key k in the first empty slot at a certain index $c(k, i)$ found while probing the HT

Open Addressing (cont'd)

EXAMPLE: Search for key k

- 1 Let $i = 0$
- 2 Compute $c(k, i)$
- 3 Probe slot $c(k, i)$: compare the slot at index $c(k, i)$ of the HT with k
- 4 If the slot is not empty and contains the key k , stops the search ($\rightarrow$ key found)
- 5 Otherwise, increment i and ...
- 6 ... Repeat previous steps 2–5 until either an empty slot is found or $i = M$ ($\rightarrow$ key not found)

Open Addressing (cont'd)

Different approach according to how $c(k, i)$ is defined

- Linear probing

$$c(k, i) = (h(k) + i) \bmod M, \quad i = 0, 1, \dots$$

- Quadratic probing

$$c(k, i) = \lfloor h(k) + c_1 \cdot i + c_2 \cdot i^2 \rfloor \bmod M, \quad i = 0, 1, \dots$$

- Double hashing

$$c(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod M, \quad i = 0, 1, \dots$$

- ...

Open Addressing (cont'd)

We focus on Linear Probing

$$c(k, i) = (h(k) + i) \bmod M, \quad i = 0, 1, \dots$$

Open Addressing (cont'd)

Issues

- ❶ Hash Table may become full
- ❷ Deletion is problematic because may leave “holes”

Open Addressing (cont'd)

Issue #1: HT may become full

- Since collisions are stored directly in the HT, it is not possible to store a number of keys greater than capacity M of the HT
 - This issue does not happen with Separate Chaining since collisions are stored in linked lists
- Also, the closer is the number of stored keys to the capacity of the HT, the larger is the number of collisions (and the worst is the performance of the HT due to a higher number of slots to probe)

Open Addressing (cont'd)

Issue #1: HT may become full → Solution

- Use the **resizing+rehashing** technique
 - To avoid too much collisions and to have enough space (i.e., empty slots) where to store new keys, dynamically *increase* the capacity of the HT when the number of stored keys is exceeding a fraction of the total number of slots
 - Typically, when $\alpha \geq 0.5$
 - To save space, dynamically *decrease* the capacity of the HT when the number of stored keys is below a fraction of the total number of slots
 - Typically, when $\alpha \leq 0.125$
 - Keys must be *rehashed* since their hash value has changed (due to the new capacity of the HT)
 - For instance, if M_1 and M_2 are the old and new HT capacities, and we use the division method as hash function, in general:

$$(k \bmod M_1) \neq (k \bmod M_2)$$

Open Addressing (cont'd)

Issue #2: Deletion is problematic because may leave “holes”

- When we delete a key from slot j , we cannot simply set that slot to empty
- If we did, we might be unable to retrieve any key k during whose insertion we had probed slot j and found it occupied

Open Addressing (cont'd)

EXAMPLE: The problem of holes left by deletions

- Insertions of colliding keys k_1, k_2, k_3 (i.e., $h(k_1) = h(k_2) = h(k_3)$):
 - Insert k_1 at index 2 ($c(k_1, 0) = 2$)
 - Insert k_2 at index 3 (collision at index 2:
 $c(k_2, 0) = 2, c(k_2, 1) = 3$)
 - Insert k_3 at index 4 (collisions at indices 2, 3:
 $c(k_3, 0) = 2, c(k_3, 1) = 3, c(k_3, 2) = 4$)
- Delete k_2 (hole at index 3)
- Search for k_3 :
 - probe index 2: slot not empty and key different
 - probe index 3: slot empty $\rightarrow$ key not found $\rightarrow$ **NO!**

Open Addressing (cont'd)

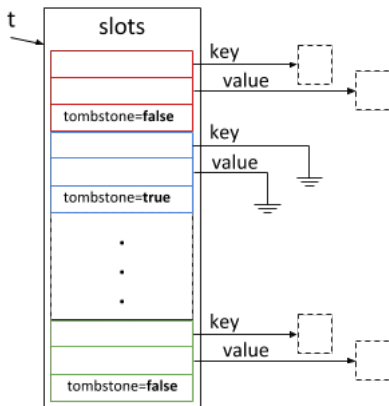
Deletion is problematic → Solution

- Use the **tombstone** trick
 - Use a special value to mark the slot as “deleted” to distinguish it from an a regular empty slot
 - This marked slot is sometimes called **tombstone**
 - Future insertion can write in this slot, but lookups treat it as a collision (i.e., the search continues in the next slot).

Open Addressing (cont'd)

Representation of a HT with Open Addressing

- slots is an array of entries, each of which stores a pointer to a **key**, a pointer to the associated **value**, and a boolean flag **tombstone** (used for the *tombstone* trick)



Open Addressing (cont'd)

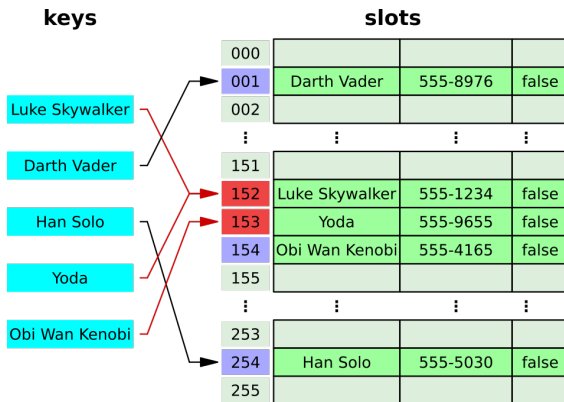


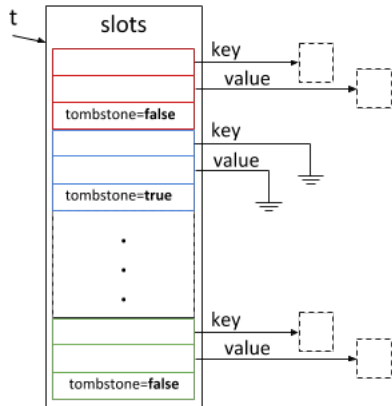
Image source: Inspired by "Hash table 5 0 1 1 1 0 SP" by Jorge Stolfi – Own work. Licensed under CC BY-SA 3.0 via

Wikimedia Commons – https://commons.wikimedia.org/wiki/File:Hash_table_5_0_1_1_1_0_SP.svg

Open Addressing (cont'd)

Let t be a hash table,

- $t[j]$: the slot of t at index j
- $\text{CAPACITY}(t)$: the total number of slots in t
- $\text{LOADFACTOR}(t)$: the load factor of t
- $\text{SIZE}(t)$: the number of stored keys in t
- $\text{KEY}(t[j])$: the key stored in slot j
- $\text{VALUE}(t[j])$: the value stored in slot j
- $\text{TOMBSTONE}(t[j])$: the flag used to mark as tombstone the slot j
- $\text{HASH}(k)$: the hash value of k
- $t[j]$ is an *empty slot* if $\text{KEY}(t[j]) = \text{null}$ and $\text{TOMBSTONE}(t[j]) = \text{false}$
- $t[j]$ is a *tombstone slot* if $\text{TOMBSTONE}(t[j]) = \text{true}$



Open Addressing (cont'd)

Assumptions and invariants that must be assured:

- Valid keys cannot be **null**
- Use the resizing+rehashing techniques
 - There will always be at least an empty slot in the HT
- Use the tombstone trick
 - Deletion at slot j always sets to **null** both the key and the value field of the slot and marks the slot as a tombstone, that is:

$$\text{KEY}(t[j]) \leftarrow \text{null},$$
$$\text{VALUE}(t[j]) \leftarrow \text{null},$$
$$\text{TOMBSTONE}(t[j]) \leftarrow \text{true}.$$

- An empty slot is initialized with a **null** value for both the key and the value and with a **false** value for the tombstone field

Open Addressing (cont'd)

The $\text{get}(t, k)$ operation: searches a key and returns the associated value

Probe slots until one of the following conditions occurs:

- A slot containing the searched key is found
→ Return the associated value
- An empty slot is found
→ Return **null** (key not found)
- All slots have been probed without finding neither an empty slot nor a slot with the same key
→ Impossible thanks to the resizing+rehashing technique (which resizes the HT)

Open Addressing (cont'd)

The $\text{get}(t, k)$ operation. . . (cont'd)

```
1: function GET( $t, k$ )
2:    $h \leftarrow \text{HASH}(k)$  /* The  $c(k, 0)$  function */
3:   while (  $\text{KEY}(t[h]) \neq \text{null}$  and  $k \neq \text{KEY}(t[h])$  ) or
       $\text{TOMBSTONE}(t[h]) = \text{true}$  do
4:      $h \leftarrow (h + 1) \bmod \text{CAPACITY}(t)$  /* The  $c(k, i)$  function */
5:   end while
6:   if  $\text{KEY}(t[h]) \neq \text{null}$  then
7:     return  $\text{VALUE}(t[h])$ 
8:   else
9:     return null
10:  end if
11: end function
```

Open Addressing (cont'd)

The `contains(t,k)` operation: like the `get(t,k)` operation, but returns true or false

```
1: function CONTAINS( $t, k$ )
2:    $h \leftarrow \text{HASH}(k)$  /* The  $c(k, 0)$  function */
3:   while (  $\text{KEY}(t[h]) \neq \text{null}$  and  $k \neq \text{KEY}(t[h])$  ) or
       $\text{TOMBSTONE}(t[h]) = \text{true}$  do
4:      $h \leftarrow (h + 1) \bmod \text{CAPACITY}(t)$  /* The  $c(k, i)$  function */
5:   end while
6:   if  $\text{KEY}(t[h]) \neq \text{null}$  then
7:     return true
8:   else
9:     return false
10:  end if
11: end function
```

Open Addressing (cont'd)

The $\text{put}(t, k, v)$ operation: inserts a new key-value pair or updates the value of an existing key, and returns the previously stored value (in case of an update) or `null` (in case of an insertion)

Probe slots until one of the following conditions occurs:

- A slot containing the same key is found
→ Update here
- An empty slot is found before finding some tombstone slot
→ Insert here
- An empty slot is found after finding some tombstone slot
→ Insert in the first tombstone slot encountered during probing
- All slots have been probed without finding a slot where to put the new key-value pair
→ Impossible thanks to the resizing+rehashing technique (which resizes the HT)

Open Addressing (cont'd)

The $\text{put}(t, k, v)$ operation... (cont'd)

```
1: function PUT( $t, k, v$ )
2:    $v_{old} \leftarrow \text{null}$ 
3:   if LOADFACTOR( $t$ )  $\geq 0.5$  then
4:     RESIZE( $t, \text{CAPACITY}(t) * 2$ )
5:   end if
6:    $h \leftarrow \text{HASH}(k)$ 
7:    $\text{found}_{tomb} \leftarrow \text{false}$ 
8:   while (  $\text{KEY}(t[h]) \neq \text{null}$  and  $k \neq$ 
    $\text{KEY}(t[h])$  ) or  $\text{TOMBSTONE}(t[h]) = \text{true}$ 
   do
9:     if  $\text{TOMBSTONE}(t[h])$  and not
        $\text{found}_{tomb}$  then
10:       $\text{found}_{tomb} \leftarrow \text{true}$ 
11:       $h_{tomb} \leftarrow h$ 
12:    end if
13:     $h \leftarrow (h + 1) \bmod \text{CAPACITY}(t)$ 
14:  end while
15:  if  $\text{KEY}(t[h]) = \text{null}$  then
16:    if  $\text{found}_{tomb}$  then
17:       $h \leftarrow h_{tomb}$ 
18:    end if
19:     $\text{KEY}(t[h]) \leftarrow k$ 
20:     $\text{VALUE}(t[h]) \leftarrow v$ 
21:     $\text{TOMBSTONE}(t[h]) \leftarrow \text{false}$ 
22:  else
23:     $v_{old} \leftarrow \text{VALUE}(t[h])$ 
24:     $\text{VALUE}(t[h]) \leftarrow v$ 
25:  end if
26:  return  $v_{old}$ 
27: end function
```

Open Addressing (cont'd)

The $\text{insert}(t, k, v)$ operation: like the $\text{put}(t, k, v)$ operation, but ignores duplicates

Probe slots until one of the following conditions occurs:

- A slot containing the same key is found
→ Do nothing
- An empty slot is found before finding some tombstone slot
→ Insert here
- An empty slot is found after finding some tombstone slot
→ Insert in the first tombstone slot encountered during probing
- All slots have been probed without finding a slot where to put the new key-value pair
→ Impossible thanks to the resizing+rehashing technique (which resizes the HT)

Open Addressing (cont'd)

The `insert(t,k,v)` operation... (cont'd)

```
1: procedure INSERT(t, k, v)
2:   if LOADFACTOR(t) ≥ 0.5 then
3:     RESIZE(t, CAPACITY(t)*2)
4:   end if
5:   h ← HASH(k)
6:   foundtomb ← false
7:   while ( KEY(t[h]) ≠ null and k ≠
  KEY(t[h]) ) or TOMBSTONE(t[h]) = true
  do
8:     if TOMBSTONE(t[h]) and not
foundtomb then
9:       foundtomb ← true
10:      htomb ← h
11:    end if
12:    h ← (h + 1) mod CAPACITY(t)
13:  end while
14:  if KEY(t[h]) = null then
15:    if foundtomb then
16:      h ← htomb
17:    end if
18:    KEY(t[h]) ← k
19:    VALUE(t[h]) ← v
20:    TOMBSTONE(t[h]) ← false
21:  end if
22: end procedure
```

Open Addressing (cont'd)

The `delete(t,k)` operation: removes a key and the associated value

Probe slots until one of the following conditions occurs:

- A slot containing the searched key is found
→ Delete the key-value pair
- An empty slot is found
→ Do nothing (key not found)
- All slots have been probed without finding neither an empty slot nor a slot with the same key
→ Impossible thanks to the resizing+rehashing technique (which resizes the HT)

Open Addressing (cont'd)

The delete(t, k) operation... (cont'd)

```
1: procedure DELETE( $t, k$ )
2:    $h \leftarrow \text{HASH}(k)$ 
3:   while (  $\text{KEY}(t[h]) \neq \text{null}$  and  $k \neq \text{KEY}(t[h])$  ) or  $\text{TOMBSTONE}(t[h]) = \text{true}$  do
4:      $h \leftarrow (h + 1) \bmod \text{CAPACITY}(t)$ 
5:   end while
6:   if  $\text{KEY}(t[h]) \neq \text{null}$  then
7:      $\text{KEY}(t[h]) \leftarrow \text{null}$ 
8:      $\text{VALUE}(t[h]) \leftarrow \text{null}$ 
9:      $\text{TOMBSTONE}(t[h]) \leftarrow \text{true}$ 
10:    if  $\text{LOADFACTOR}(t) \leq 0.125$  then
11:       $\text{RESIZE}(t, \text{CAPACITY}(t)/2)$ 
12:    end if
13:  end if
14: end procedure
```

The Hash Table ADT `upo_ht_*_t`

Implementation requirements:

- Keys and values must be of generic type
→ Use `void*`
- To compute hash values, we need a *key hash function*

```
size_t hash(const void *k, size_t m);
```

which takes two parameters k (the key) and m (the capacity of the HT), and returns the hash value of k

→ Use a **hasher** (a pointer to a key hash function):

```
typedef size_t (*upo_ht_hasher_t)(const void*, size_t);
```

and store it in the HT data structure

The Hash Table ADT `upo_ht_*_t`

Some ready-to-use hash functions (see `include/upo/hashtable.h`):

- For integers:
 - `upo_ht_hash_int_div()`: division method
 - `upo_ht_hash_int_mult_knuth()`: a variant of the multiplication method (by Knuth)
 - ...
- For strings:
 - `upo_ht_hash_str_kr2e()`: variant of the modular sum of the integer representation of characters (by Kernighan and Ritchie)
 - `upo_ht_hash_str_java()`: variant of the modular sum of the integer representation of characters (used by Java)
 - ...

The Hash Table ADT `upo_ht_*_t` (cont'd)

Implementation requirements:

- To compare keys, we need a *key comparison function*

```
int compare(const void *a, const void *b);
```

which takes two parameters *a* and *b*, and returns a value:

- < 0 if $*a < *b$
- $= 0$ if $*a = *b$
- > 0 if $*a > *b$

NOTE: $*a, *b$ is only a shorthand notation since **void*** cannot be dereferenced

→ Use a **comparator** (a pointer to a key comparison function):

```
typedef int (*upo_ht_comparator_t)(const void*, const void*);
```

and store it in the HT data structure

The Hash Table ADT `upo_ht_*_t` (cont'd)

Implementation requirements:

- When an element is removed or the ADT is destroyed, who takes care of freeing memory allocated for user data?
→ Use an additional parameter `destroy_data` to let the user decide:

```
void upo_ht_..._clear(..., int destroy_data);  
void upo_ht_..._destroy(..., int destroy_data);  
void upo_ht_..._delete(..., int destroy_data);
```

If `destroy_data` is $\neq 0$ the memory allocated for user-data (i.e., pointers to keys and values) is deallocated (with the `free()` function)

The Hash Table ADT `upo_ht_*_t` (cont'd)

Implementation requirements:

- The `upo_ht_..._size(...)` operation must scan every slot of the HT to count stored keys
- A way to speed-up this operation is to keep track of the number of stored keys every time the HT is changed
- To do so, the HT data structure contains the `size` field that can be used for this purpose
- This field *must* be updated every time a key is inserted into or removed from the HT

The Hash Table ADT

The HT with Separate Chaining ADT `upo_ht_sepchain_t`

```
1  struct upo_ht_sepchain_list_node_s { // Node of the list of  
    collisions  
2      void *key;  
3      void *value;  
4      struct upo_ht_sepchain_list_node_s *next;  
5  };  
6  typedef struct upo_ht_sepchain_list_node_s  
    upo_ht_sepchain_list_node_t;  
7  struct upo_ht_sepchain_slot_s { // Slot of HT  
8      upo_ht_sepchain_list_node_t *head;  
9  };  
10 typedef struct upo_ht_sepchain_slot_s upo_ht_sepchain_slot_t;  
11 struct upo_ht_sepchain_s { // HT with separate chaining  
12     upo_ht_sepchain_slot_t *slots; // Array of slots.  
13     size_t capacity; // The capacity of the HT.  
14     size_t size; // The number of stored key-value pairs.  
15     upo_ht_hasher_t key_hash; // The key hash function.  
16     upo_ht_comparator_t key_cmp; // The key comparison function.  
17 };  
18 typedef struct upo_ht_sepchain_s* upo_ht_sepchain_t;
```

The Binary Search Tree ADT

The HT with Separate Chaining ADT `upo_ht_sepchain_t` (cont'd)

How to use the `upo_ht_sepchain_t` ADT?

```
1  #include <upo/hashtable.h>
2  int int_cmp(const void *a, const void *b) {
3      const int *aa = a, *bb = b; return (*aa > *bb) - (*aa < *bb);
4  }
5  ...
6  upo_ht_sepchain_t ht = NULL;
7  int keys[] = {2,1,0,3};
8  char *values[] = {"two","one","zero","three"};
9  size_t i;
10 ht = upo_ht_sepchain_create(UPO_HT_SEPCCHAIN_DEFAULT_CAPACITY, upo_ht_hash_int_div, int_cmp);
11 for (i = 0; i < sizeof keys/sizeof keys[0]; ++i)
12     upo_ht_sepchain_insert(ht, &keys[i], &values[i]);
13 int xkey = keys[2];
14 if (upo_ht_sepchain_contains(ht, &xkey)) {
15     printf("Found: %d -> %s\n", xkey, *(((char**) upo_ht_sepchain_get(ht, &keys[2]))));
16     upo_ht_sepchain_delete(ht, &xkey, 0);
17 } else {
18     printf("Key %d not found\n", xkey);
19 }
20 upo_ht_sepchain_destroy(ht, 0);
```

The HT with Linear Probing ADT `upo_ht_linprob_t`

```
1  struct upo_ht_linprob_slot_s { // Slot of HT
2      void *key;
3      void *value;
4      int tombstone; // Flag used to mark this slot as deleted.
5  };
6  typedef struct upo_ht_linprob_slot_s upo_ht_linprob_slot_t;
7
8  struct upo_ht_linprob_s { // Type for HTs with linear probing
9      upo_ht_linprob_slot_t *slots; // Array of slots
10     size_t capacity; // The capacity of the HT.
11     size_t size; // The number of stored key-value pairs.
12     upo_ht_hasher_t key_hash; // The key hash function.
13     upo_ht_comparator_t key_cmp; // The key comparison function.
14 };
15 typedef struct upo_ht_linprob_s* upo_ht_linprob_t;
```

The HT with Linear Probing ADT `upo_ht_linprob_t` (cont'd)

How to use the `upo_ht_linprob_t` ADT?

```
1  #include <upo/hashtable.h>
2  int int_cmp(const void *a, const void *b) {
3      const int *aa = a, *bb = b; return (*aa > *bb) - (*aa < *bb);
4  }
5  ...
6  upo_ht_linprob_t ht = NULL;
7  int keys[] = {2,1,0,3};
8  char *values[] = {"two","one","zero","three"};
9  size_t i;
10 ht = upo_ht_linprob_create(UPO_HT_LINPROB_DEFAULT_CAPACITY, upo_ht_hash_int_div, int_cmp);
11 for (i = 0; i < sizeof keys/sizeof keys[0]; ++i)
12     upo_ht_linprob_insert(ht, &keys[i], &values[i]);
13 int xkey = keys[2];
14 if (upo_ht_linprob_contains(ht, &xkey)) [
15     printf("Found: %d -> %s\n", xkey, *((char**) upo_ht_linprob_get(ht, &keys[2])));
16     upo_ht_linprob_delete(ht, &xkey, 0);
17 ] else {
18     printf("Key %d not found\n", xkey);
19 }
20 upo_ht_linprob_destroy(ht, 0);
```

Exercise #1: The HT ADT `upo_ht_sepchain_t`

Tasks:

- 1 Prepare your environment
- 2 Implement the Hash Table ADT `upo_ht_sepchain_t`
- 3 Test your implementation

Exercise #1: The HT ADT `upo_ht_sepchain_t` (cont'd)

Task #1 – Prepare Your Environment:

- Make sure you have already extracted the file `UP0alglib_core_toolkit.zip`
- Move to the directory `$ADT_PATH`
- Download and extract the file `hashtable-ex01_02_03-adt.zip`

```
1 include/upo/hashtable.h
2 src/hashtable_private.h
3 src/hashtable.c
4 test/mk/hashtable.mk
5 test/test_hashtable_linprob.c
6 test/test_hashtable_linprob_more.c
7 test/test_hashtable_sepchain.c
8 test/test_hashtable_sepchain_more.c
```

Exercise #1: The HT ADT `upo_ht_sepchain_t` (cont'd)

Task #2 – Implement ADT for HT with Separate Chaining:

- Implements functions inside the file `src/hashtable.c` that are enclosed by comments:

```
1  /***** EXERCISE #1 - BEGIN of HASH TABLE with SEPARATE CHAINING  
   *****/  
2  ...  
3  /***** EXERCISE #1 - END of HASH TABLE with SEPARATE CHAINING  
   *****/
```

and only those that contain the comment:

```
1  /* TO STUDENTS:  
2  ...  
3  */
```

- Compile: `make clean bin`

Exercise #1: The HT ADT `upo_ht_sepchain_t` (cont'd)

Task #3 – Test Your Implementation:

```
1  make clean test
2  ./test/test_hashtable_sepchain
3  valgrind --tool=memcheck --leak-check=full
    ./test/test_hashtable_sepchain
```

Exercise #1: The HT ADT `upo_ht_sepchain_t` (cont'd)

Output of `./test/test_hashtable_sepchain`

```
1 Test case 'create/destroy'... OK
2 Test case 'put/get/delete'... OK
3 Test case 'insert/get/delete'... OK
4 Test case 'clear'... OK
5 Test case 'empty'... OK
6 Test case 'size'... OK
7 Test case 'hash_funcs'... OK
8 Test case 'null'... OK
```

Exercise #1: The HT ADT `upo_ht_sepchain_t` (cont'd)

Output of `valgrind --tool=memcheck --leak-check=full`

```
./test/test_hashtable_sepchain
```

```
1  ...
```

```
2
```

```
3  All heap blocks were freed -- no leaks are possible
```

```
4
```

```
5  For counts of detected and suppressed errors, rerun with: -v
```

```
6  ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from  
    0)
```

Exercise #2: The HT ADT `upo_ht_linprob_t`

Tasks:

- 1 Prepare your environment
- 2 Implement the Hash Table ADT `upo_ht_linprob_t`
- 3 Test your implementation

Exercise #2: The HT ADT `upo_ht_linprob_t` (cont'd)

Task #1 – Prepare Your Environment:

- Make sure you have already extracted the file `UP0alglib_core_toolkit.zip`
- Make sure you have already extracted the file `hashtable-ex01_02_03-adt.zip`
- Make sure you have already completed Exercise #1
- Move to the directory `$ADT_PATH`

Exercise #2: The HT ADT `upo_ht_linprob_t` (cont'd)

Task #2 – Implement ADT for HT with Linear Probing:

- Implements functions inside the file `src/hashtable.c` that are enclosed by comments:

```
1  /* **** EXERCISE #2 - BEGIN of HASH TABLE with LINEAR PROBING  
    ****/  
2  ...  
3  /* **** EXERCISE #2 - END of HASH TABLE with LINEAR PROBING ****/
```

and only those that contain the comment:

```
1  /* TO STUDENTS:  
2  ...  
3  */
```

- Compile: `make clean bin`

Exercise #2: The HT ADT `upo_ht_linprob_t` (cont'd)

Task #3 – Test Your Implementation:

```
1  make clean test
2  ./test/test_hashtable_linprob
3  valgrind --tool=memcheck --leak-check=full
    ./test/test_hashtable_linprob
```

Exercise #2: The HT ADT `upo_ht_linprob_t` (cont'd)

Output of `./test/test_hashtable_linprob`

```
1 Test case 'create/destroy'... OK
2 Test case 'put/get/delete'... OK
3 Test case 'insert/get/delete'... OK
4 Test case 'clear'... OK
5 Test case 'empty'... OK
6 Test case 'size'... OK
7 Test case 'resize'... OK
8 Test case 'hash_funcs'... OK
9 Test case 'null'... OK
```

Exercise #2: The HT ADT `upo_ht_linprob_t` (cont'd)

Output of `valgrind --tool=memcheck --leak-check=full`

```
./test/test_hashtable_linprob
```

```
1  ...
```

```
2
```

```
3  All heap blocks were freed -- no leaks are possible
```

```
4
```

```
5  For counts of detected and suppressed errors, rerun with: -v
```

```
6  ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from  
    0)
```

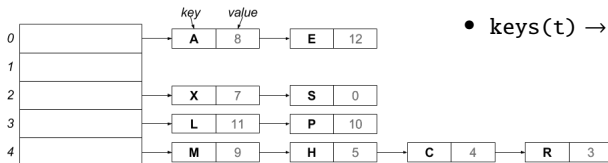
Exercise #3: The HT ADT `up0_ht*_t` – Extra Ops

Other common operations:

- `keys(t)`: returns all the keys in `t`
- `traverse(t,v)`: applies function `v` to every key-value pair stored in `t`

Exercise #3: The HT ADT `upo_ht_*_t` – Extra Ops (cont'd)

The `keys(t)` operation: return the list of all the keys stored in the HT
Separate Chaining



- `keys(t) → [A, E, X, S, L, P, M, H, C, R]`

The Hash Table ADT

Exercise #3: The HT ADT `upo_ht*_t` – Extra Ops (cont'd)

The `keys(t)` operation: returns the list of all the keys stored in the HT
Open Addressing

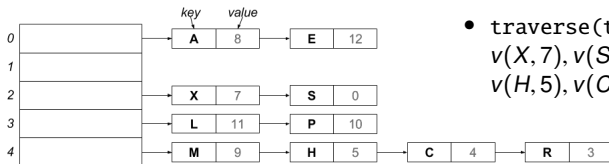
	key	value
0	P	10
1	M	9
2		
3		
4	A	8
5	C	4
6	S	0
7	H	5
8	L	11
9		
10	E	12
11		
12		
13		
14	R	3
15	X	7

- `keys(t) → [P, M, A, C, S, H, L, E, R, X]`

Exercise #3: The HT ADT `upo_ht_*_t` – Extra Ops (cont'd)

The `traverse(t, v)` operation: applies function `v` to every key-value pair stored in `t`

Separate Chaining



- `traverse(t, v) → v(A, 8), v(E, 12), v(X, 7), v(S, 0), v(L, 11), v(P, 10), v(M, 9), v(H, 5), v(C, 4), v(R, 3)`

The Hash Table ADT

Exercise #3: The HT ADT `upo_ht_*_t` – Extra Ops (cont'd)

The `traverse(t,v)` operation: applies function `v` to every key-value pair stored in `t`

Open Addressing

	key	value
0	P	10
1	M	9
2		
3		
4	A	8
5	C	4
6	S	0
7	H	5
8	L	11
9		
10	E	12
11		
12		
13		
14	R	3
15	X	7

- $\text{traverse}(t, v) \rightarrow v(P, 10), v(M, 9), v(A, 8), v(C, 4), v(S, 0), v(H, 5), v(L, 11), v(E, 12), v(R, 3), v(X, 7)$

Exercise #3: The HT ADT `upo_ht*_t` – Extra Ops (cont'd)

Tasks:

- 1 Prepare your environment
- 2 Implement the additional operation for the Hash Table ADT
- 3 Test your implementation

Exercise #3: The HT ADT `upo_ht_*_t` – Extra Ops (cont'd)

Task #1 – Prepare Your Environment:

- Make sure you have already extracted the file `UP0alglib_core_toolkit.zip`
- Make sure you have already extracted the file `hashtable-ex01_02_03-adt.zip`
- Make sure you have already completed Exercises #1 and #2
- Move to the directory `$ADT_PATH`

Exercise #3: The HT ADT `upo_ht_*_t` – Extra Ops (cont'd)

Task #2 – Implement the additional operations for the HT with Separate Chaining and with Linear Probing ADTs:

- Implements functions inside the file `src/hashtable.c` that are enclosed by comments:

```
1  /***** EXERCISE #3 - BEGIN of HASH TABLE - EXTRA OPERATIONS *****/
2  ...
3  /***** EXERCISE #3 - END of HASH TABLE - EXTRA OPERATIONS *****/
```

and only those that contain the comment:

```
1  /* TO STUDENTS:
2  ...
3  */
```

- Compile: `make clean bin`

Exercise #3: The HT ADT `upo_ht_*_t` – Extra Ops (cont'd)

Task #3 – Test Your Implementation:

```
1  make clean test
2  ./test/test_hashtable_sepchain_more
3  valgrind --tool=memcheck --leak-check=full
    ./test/test_hashtable_sepchain_more
4  ./test/test_hashtable_linprob_more
5  valgrind --tool=memcheck --leak-check=full
    ./test/test_hashtable_linprob_more
```

Exercise #3: The HT ADT `upo_ht*_t` – Extra Ops (cont'd)

Output of `./test/test_hashtable_sepchain_more`

- 1 Test case 'keys'... OK
- 2 Test case 'traverse'... OK

Output of `./test/test_hashtable_linprob_more`

- 1 Test case 'keys'... OK
- 2 Test case 'traverse'... OK